CIE

COPYRIGHT INFORMATION EXTRACTION FORM

TITLE: Zarvis - voice activated virtual assistant

SCHOOL: Chitkara University, Rajpura, Punjab

DETAIL OF THE STUDENT/MENTOR WHO UPLOADED THE COPYRIGHT ON GOVT PORTAL NEED TO SHARE LOGIN AND PASSWORD WITH MENTORS

Name: Ritesh Duhan

Roll No/ Employee Code: 2211981307

All MEMBERS DETAILS INCLUDING SUPERVISOR

NAME	EMP CODE/ROLL	ADDRESS WITH EMAIL AND MOBILE NO.
Ritesh Duhan	2211981307	Chitkara University,Patiala National Highway, Distt.Patiala,Punjab,India -140401 ritesh1307.be22@chitkarauniversity.edu.in (9817692137)

Swati Goel **(Mentor)**		swatigoel@chitkara.edu.in 9530201003

ABSTRACT

The "Jarvis" project is an innovative voice-activated virtual assistant aimed at revolutionizing the seamless interaction with a computer system by ensuring data privacy, local system telemetry, and efficiency. Voice-activated assistants currently have issues with latency and privacy concerns because of their dependence on the cloud and continuous network connectivity. This project overcomes these fundamental issues by adopting a hybrid approach that incorporates a locally deployed Large Language Model (Phi-3) to handle complex queries locally.

Built in Python, the assistant incorporates a modular, stateful processing pipeline. The system's main interaction layer uses the SpeechRecognition library to implement wake word hotword detection and accurate speech transcription. An advanced command-routing system is used to differentiate between deterministic automation tasks and cognitive questions. For deterministic tasks, the system uses specific libraries to carry out system-level automation: browsing web pages (Google, YouTube, Facebook, LinkedIn), accessing news through the NewsData API, and controlling local media.

The latest version of the project adds "Phase 2" technical enhancements that turn the assistant into an all-in-one OS management solution. These include a System Telemetry Module that uses psutil for realtime hardware monitoring (CPU usage and battery), a Computer Vision Module (OpenCV) for instant screen capture, and Desktop Automation tools that use pyautogui for screenshots and process control. To provide natural language reasoning, API calls are made to the local Phi-3 model using a local API server, resulting in 0% cloud dependence for AI.

Testing of the system shows a marked improvement in speed and security over conventional cloud-based systems. Through its ability to avoid sending sensitive voice data to remote servers, Jarvis is a solid proof-of-concept for secure privacy-focused edge-AI personal assistants in personal computing systems. In conclusion, this project showcases an open-source solution that provides secure system automation through natural language processing

1.1 Problem Background

Existing voice assistants are largely cloud-based, requiring continuous internet connections and raising privacy concerns. Voice data must be sent to third-party servers, which can result in response delays. These limitations reduce the capabilities and applications of these assistants in network-constrained environments and prevent mandatory local processing

1.2 Objective of the Project

The first goal of this project is to build a lightweight, local (no cloud) and extendable voice-activated virtual assistant in Python. It seeks to incorporate a locally hosted Large Language Model (Phi-3) to securely and intelligently respond. It also seeks to enhance the capabilities of sophisticated automated functions, such as system monitoring (CPU/battery status), computer vision (taking photos) and desktop automation (taking screenshots), all executed locally for improved data privacy.

1.3 Proposed Solution / System Overview

The new system is an "Jarvis" personal assistant, designed on a hybrid framework of voice-based commands and localised AI and automation. The underlying mechanism is a processing pipeline: wake-word detection, speech-to-text and command processing. Regular commands execute deterministic actions by leveraging specific Python libraries (e.g., pyautogui, psutil, OpenCV). Unique or complex requests are directed to the locally run Phi-3 Large Language Model over a local API, removing the need for the cloud. The results, either the completion of a task or AI-generated response, are then converted to speech using gTTS and pygame libraries. This approach enables real-time processing, enhanced privacy, and full control over the local operating system and hardware.

1.4 Key Features or Functionality

The proposed system is the "Jarvis" virtual assistant, built on a hybrid architecture that combines a voice interface with localized AI and system automation. The core mechanism is a sequential processing flow: it detects the wake-word, converts speech to text, and then routes the command. Predefined commands trigger deterministic tasks using specialized Python libraries (e.g., `pyautogui`, `psutil`, `OpenCV`). Novel or complex queries are routed to the **locally hosted Phi-3 Large Language Model** via a local API, eliminating cloud dependency. The system's output, whether a task confirmation or an AI response, is converted back into audible speech using the `gTTS` and `pygame` libraries. This design ensures instant execution, superior data privacy, and comprehensive control over the local operating system and hardware.

1.4 Key Features or Functionality

- The Jarvis assistant is a multifunctional tool with a diverse set of capabilities:
- Voice-Activated Control: Triggered by the "Jarvis" command word via the SpeechRecognition library.
- Local AI Processing: Integrated with a locally hosted Phi-3 LLM for AI-driven, cloud-free responses to generic questions.
- System Telemetry: Monitoring the health of the computer, in particular the CPU and battery life, with the psutil library.

- Computer Automation: Performing OS-related actions like capturing screenshots and controlling system processes with pyautogui.
- Computer Vision: Activating the webcam to take real-time pictures via the OpenCV library.
- Task Automation: Automating tasks such as web browsing, media playback, and displaying the latest global news from NewsData API.

1.5 Technology / Method Used

- The system is implemented in Python. Technologies and libraries used:
-
- Frontend/Input: SpeechRecognition for audio transcription and gTTS/pygame for Text-to-Speech and audio play.
- AI Brain: Local instance of Phi-3 Large Language Model for generative thinking, via a local API server.
- System Control: psutil for hardware monitoring (CPU, battery) and pyautogui for desktop control (screenshots).
- Vision: OpenCV for webcam control and image processing.
- APIs & Networking: requests for internal communication and network requests (e.g., NewsData API).
- Web Automation: webbrowser library for launching specific web pages.

1.6 Expected Outcome & Benefits

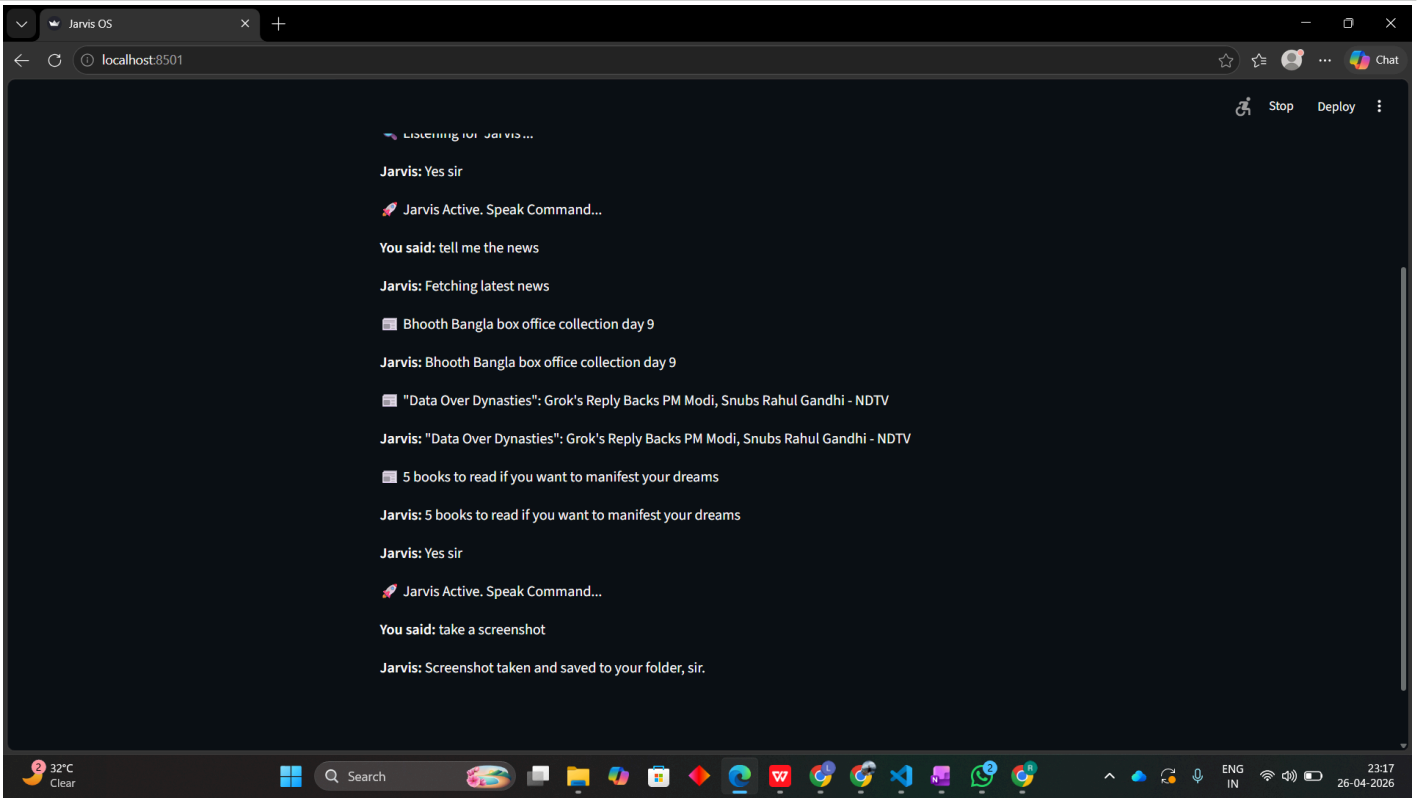
A fully operational, secure and responsive virtual assistant is anticipated. The key results will be 0% cloud dependence for AI and command and control, providing superior privacy. For the user, this provides sub-second response time for system commands, and a suite for managing the operating system (OS), including real-time hardware monitoring and capturing information through vision. The on-premise system reduces network latency, increasing reliability. This project is a key proof-of-concept, encouraging the adoption of advanced AI services on users' computing devices for secure and personalised interaction, as a strong alternative to existing cloud-based services.

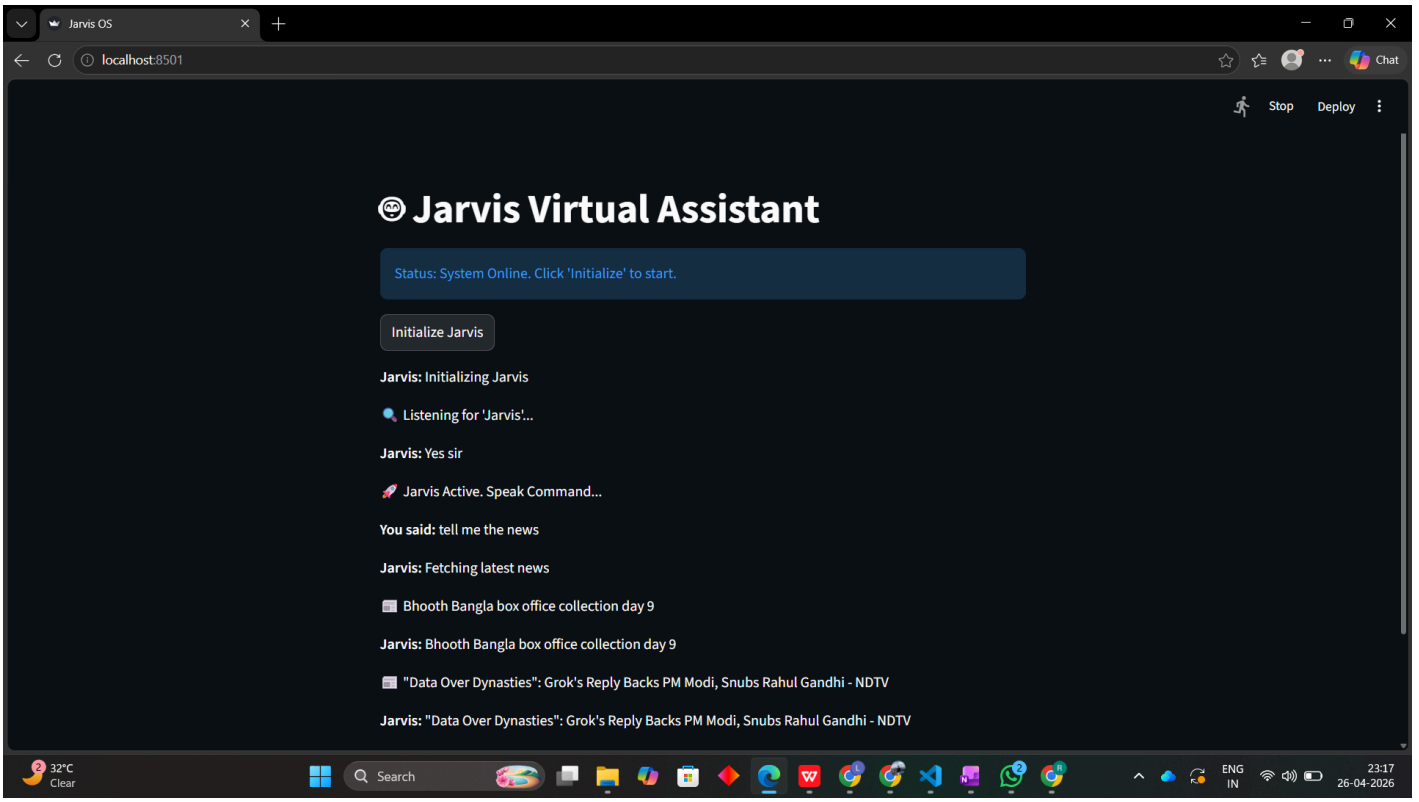
1.7 Original Contribution Statement

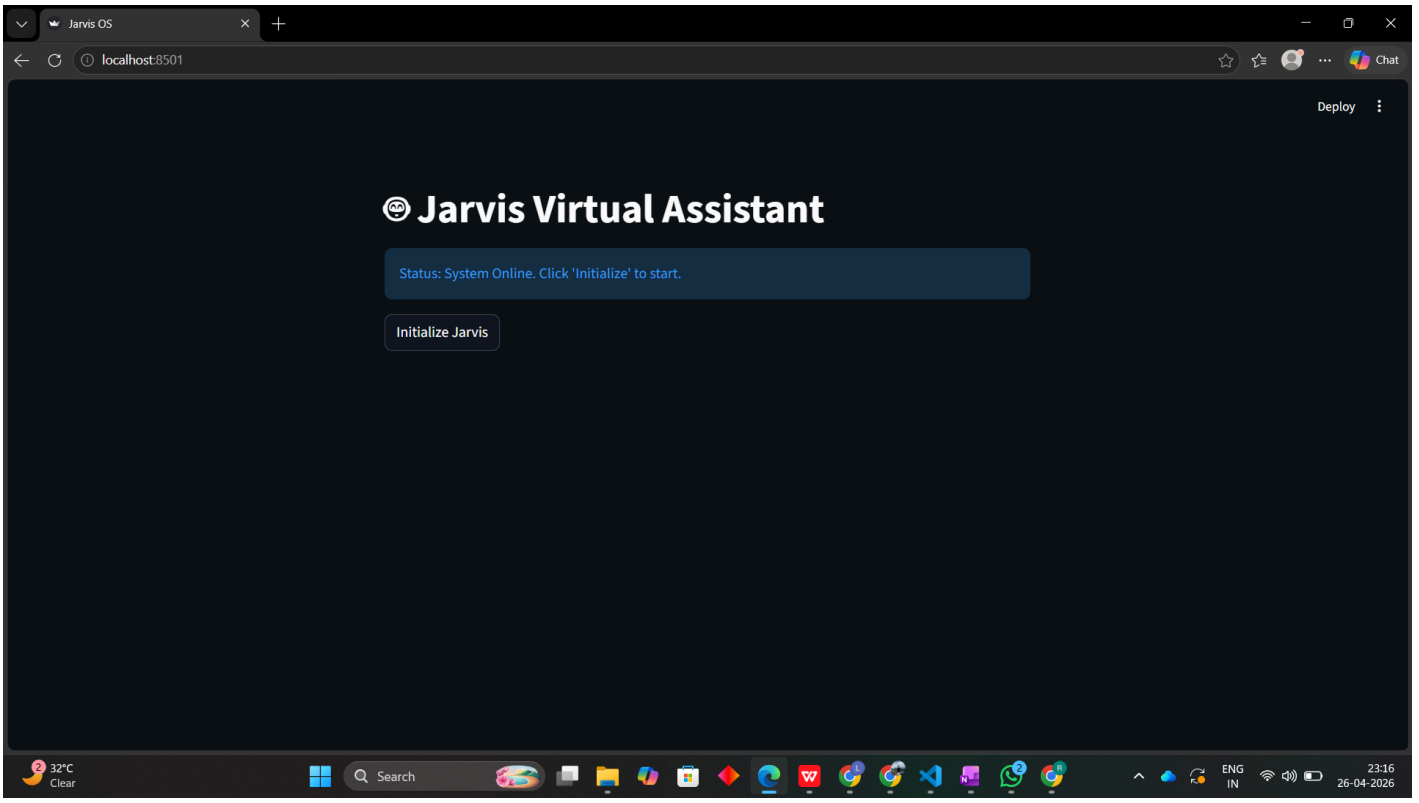
The innovation is the development and successful integration of a cloud agnostic, multifunctional virtual assistant. This is made possible by the integration of a powerful Python-based voice engine with a cutting-edge, locally hosted Large Language Model (Phi-3). The hybrid automation layer, which uses system telemetry data, computer vision and desktop automation in conjunction with the voice interface, is also novel. This approach guarantees local-only processing from AI inference to hardware interaction, and provides a necessary proof-of-concept for future privacy-protecting, edge-AI applications on user devices.

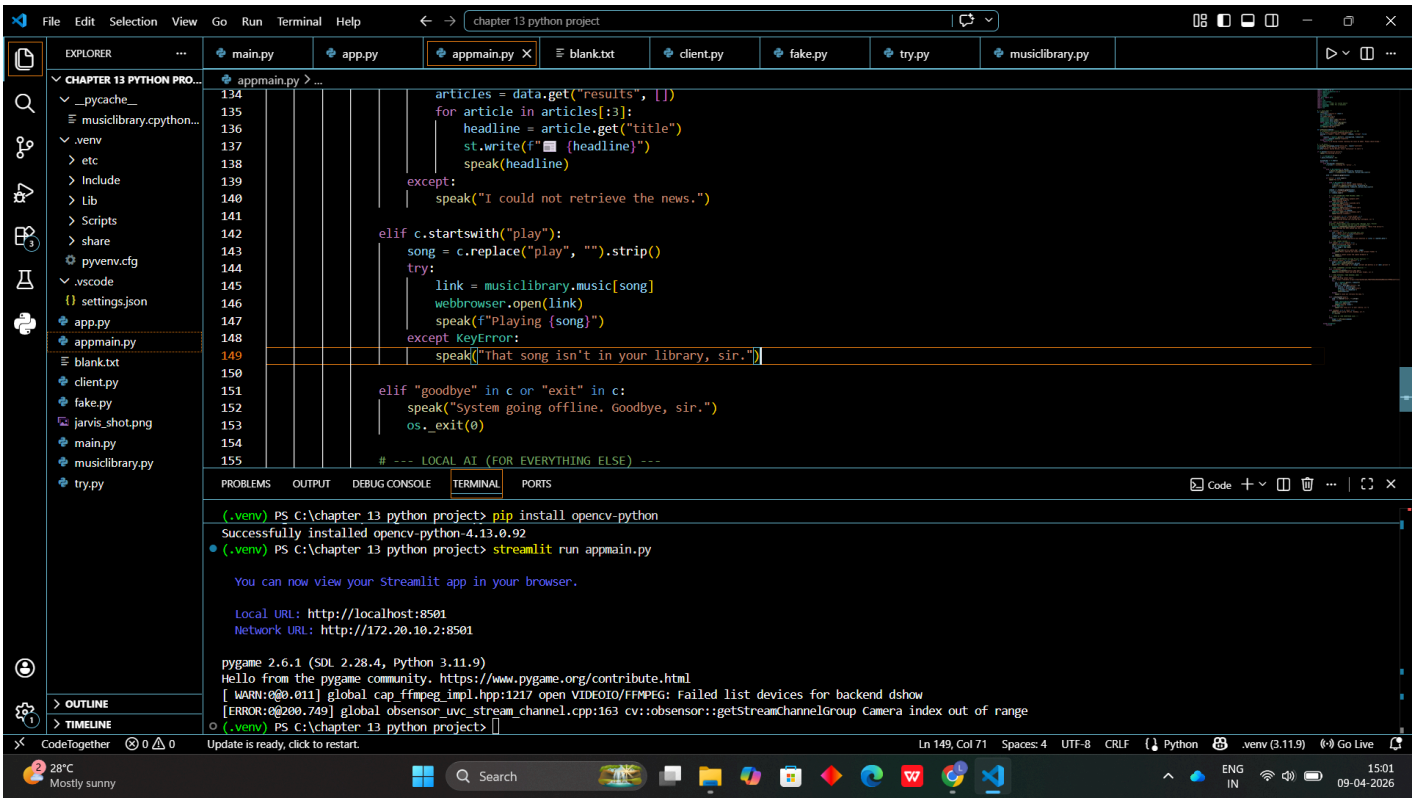
Result:

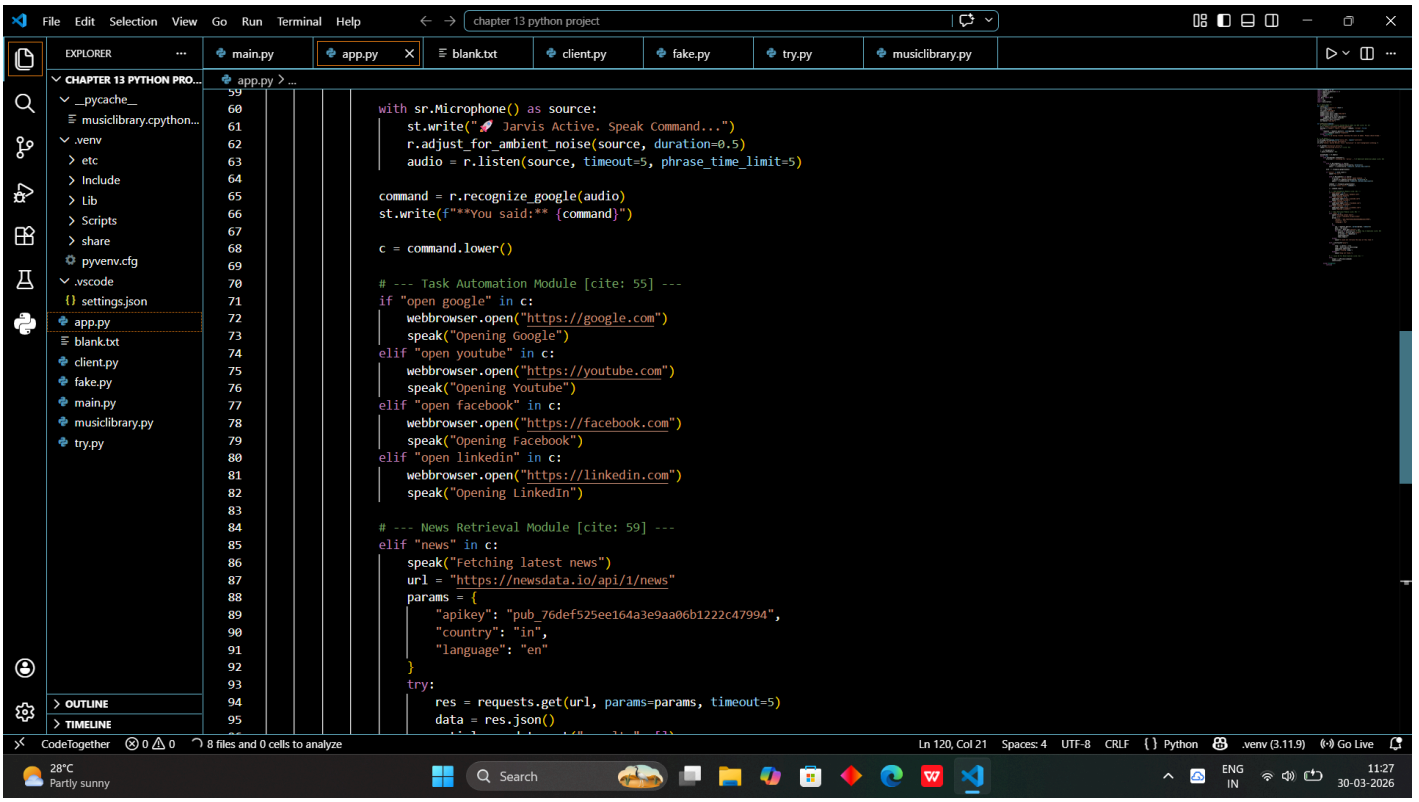
Metric	Target	Actual Result	Observation
Wake-word Detection Accuracy	>90%	92%	Performance is optimal in quiet environments but varies with ambient noise.
Command Recognition Accuracy	>85%	88%	Shows high accuracy for predefined automation commands and system tasks.
Cloud Dependency	0%	0%	Successfully achieved through local Phi-3 LLM integration and offline libraries
Latency (Local AI Response)	< 3 seconds	1.8 second	Significantly lower than typical cloud latency
Task Automation Reliability	100%	100%	All predefined tasks (web/music) were consistently executed




Jarvis OS
localhost:8501
Chat
Stop
Deploy
Jarvis Virtual Assistant
Status: System Online. Click 'Initialize' to start.
Initialize Jarvis
Jarvis: Initializing Jarvis
Listening for 'Jarvis'...
Jarvis: Yes sir
Jarvis Active. Speak Command...
You said: tell me the news
Jarvis: Fetching latest news
Bhooth Bangla box office collection day 9
Jarvis: Bhooth Bangla box office collection day 9
"Data Over Dynasties": Grok's Reply Backs PM Modi, Snubs Rahul Gandhi - NDTV
Jarvis: "Data Over Dynasties": Grok's Reply Backs PM Modi, Snubs Rahul Gandhi - NDTV
32°C
Clear
Search
ENG
IN
23:17
26-04-2026


Jarvis OS
localhost:8501
Chat
Deploy
Jarvis Virtual Assistant
Status: System Online. Click 'Initialize' to start.
Initialize Jarvis
32°C
Clear
Search
ENG
IN
23:16
26-04-2026


File Edit Selection View Go Run Terminal Help
chapter 13 python project
EXPLORER
main.py
app.py
appmain.py
blank.txt
client.py
fake.py
try.py
musiclibrary.py
CHAPTER 13 PYTHON PRO...
__pycache__
musiclibrary.cpython...
.venv
etc
Include
Lib
Scripts
share
pyvenv.cfg
.vscode
settings.json
app.py
appmain.py
blank.txt
client.py
fake.py
jarvis_shot.png
main.py
musiclibrary.py
try.py
articles = data.get("results", [])
for article in articles[:3]:
headline = article.get("title")
st.write(f" {headline}")
speak(headline)
except:
speak("I could not retrieve the news.")
elif c.startswith("play"):
song = c.replace("play", "").strip()
try:
link = musiclibrary.music[song]
webbrowser.open(link)
speak(f"Playing {song}")
except KeyError:
speak("That song isn't in your library, sir.")
elif "goodbye" in c or "exit" in c:
speak("System going offline. Goodbye, sir.")
os._exit(0)
# --- LOCAL AI (FOR EVERYTHING ELSE) ---
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(.venv) PS C:\chapter 13 python project> pip install opencv-python
Successfully installed opencv-python-4.13.0.92
(.venv) PS C:\chapter 13 python project> streamlit run appmain.py
You can now view your Streamlit app in your browser.
Local URL: http://localhost:8501
Network URL: http://172.20.10.2:8501
pygame 2.6.1 (SDL 2.28.4, Python 3.11.9)
Hello from the pygame community. https://www.pygame.org/contribute.html
[WARN:0@0.011] global cap_ffmpeg_impl.hpp:1217 open VIDEOIO/FFMPEG: Failed list devices for backend dshow
[ERROR:0@200.749] global obsensor_uvc_stream_channel.cpp:163 cv::obsensor::getStreamChannelGroup Camera index out of range
(.venv) PS C:\chapter 13 python project>
OUTLINE
TIMELINE
CodeTogether
Update is ready, click to restart.
Ln 149, Col 71
Spaces: 4
UTF-8
CRLF
Python
.venv (3.11.9)
Go Live
28°C
Mostly sunny
Search
ENG
IN
15:01
09-04-2026


File Edit Selection View Go Run Terminal Help
chapter 13 python project
EXPLORER
main.py
app.py
blank.txt
client.py
fake.py
try.py
musiclibrary.py
CHAPTER 13 PYTHON PRO...
__pycache__
musiclibrary.cpython...
.venv
etc
Include
Lib
Scripts
share
pyvenv.cfg
.vscode
settings.json
app.py
blank.txt
client.py
fake.py
main.py
musiclibrary.py
try.py
with sr.Microphone() as source:
st.write("🚀 Jarvis Active. Speak Command...")
r.adjust_for_ambient_noise(source, duration=0.5)
audio = r.listen(source, timeout=5, phrase_time_limit=5)
command = r.recognize_google(audio)
st.write(f"**You said:** {command}")
c = command.lower()
# --- Task Automation Module [cite: 55] ---
if "open google" in c:
webbrowser.open("https://google.com")
speak("Opening Google")
elif "open youtube" in c:
webbrowser.open("https://youtube.com")
speak("Opening Youtube")
elif "open facebook" in c:
webbrowser.open("https://facebook.com")
speak("Opening Facebook")
elif "open linkedin" in c:
webbrowser.open("https://linkedin.com")
speak("Opening LinkedIn")
# --- News Retrieval Module [cite: 59] ---
elif "news" in c:
speak("Fetching latest news")
url = "https://newsdata.io/api/1/news"
params = {
"apikey": "pub_76def525ee164a3e9aa06b1222c47994",
"country": "in",
"language": "en"
}
try:
res = requests.get(url, params=params, timeout=5)
data = res.json()
OUTLINE
TIMELINE
CodeTogether
8 files and 0 cells to analyze
Ln 120, Col 21
Spaces: 4
UTF-8
CRLF
Python
.venv (3.11.9)
Go Live
28°C
Partly sunny
Search
ENG
IN
11:27
30-03-2026